

# Contrato B — API de Documentação para Ingestão (v1.0)

---

**Integração:** Base de documentação do cliente → Pipeline de ingestão RAG (agente Zapin / Azapfy)  
**Data:** 2026-09-01 **Status:** Proposta para acordo **Consumidor:** job de ingestão incremental (execução diária) **Provedor:** API de documentação do cliente

---

## 1. Objetivo

---

Este contrato define como a API de documentação do cliente **DEVE** se comportar para que o pipeline de ingestão do agente de suporte sincronize a base de conhecimento de forma **incremental, diária e segura** — sem reconstruir o índice inteiro a cada ciclo e sem risco de deleção indevida de conhecimento.

As palavras **DEVE**, **NÃO DEVE** e **PODE** seguem o sentido da RFC 2119.

O princípio central do contrato: **deleção é sempre explícita** (flag `deleted`), **nunca por omissão**. A ausência de um documento na listagem é tratada pelo consumidor como anomalia, não como remoção — um retorno parcial ou filtrado por erro **não pode** causar perda de conhecimento no índice.

---

## 2. Endpoints

---

A nomenclatura das rotas é sugestão e **PODE** ser adaptada; a **semântica** de cada uma é obrigatória.

### 2.1 GET /docs — Listagem (inventário)

Retorna o inventário **completo** da documentação: todos os documentos vivos **e** todos os tombstones dentro do período de retenção (seção 4.5). É a fonte de decisão do sync — deve ser barata (sem o campo `conteudo`).

**Resposta (200):**

```
{
  "itens": [
    {
      "id": "documentacao/frotas/cadastro-veiculos.md",
      "titulo": "Cadastro de veículos",
      "categoria": "Frotas",
      "updated_at": "2026-08-30T14:22:00Z",
      "deleted": false
    },
    {
      "id": "documentacao/frotas/aviso-rntrc.md",
      "titulo": "Avisos RNTRC",
      "categoria": "Frotas",
      "updated_at": "2026-08-28T09:10:00Z",
      "deleted": true,
      "deleted_at": "2026-08-28T09:10:00Z"
    }
  ],
  "next_cursor": "opaco-ou-null"
}
```

## 2.2 GET /docs/{id} — Documento completo

Retorna o documento com o campo `conteudo` (Markdown). O consumidor só chama esta rota para documentos que a listagem indicou como novos ou alterados.

### Resposta (200):

```
{
  "id": "documentacao/frotas/cadastro-veiculos.md",
  "titulo": "Cadastro de veículos",
  "categoria": "Frotas",
  "conteudo": "# Cadastro de veículos\n\n## Documentos obrigatórios\n...",
  "updated_at": "2026-08-30T14:22:00Z",
  "deleted": false
}
```

Para um documento deletado (tombstone), esta rota **DEVE** responder `410 Gone` (ou `404`). O consumidor não busca conteúdo de tombstones.

## 2.3 Autenticação e transporte

- HTTPS obrigatório.
- `Authorization: Bearer <token>`, com credencial **somente leitura**.
- Respostas em UTF-8, `Content-Type: application/json`.

### 3. Modelo de dados

| Campo      | Tipo     | Obrigatório                               | Regras                                                                                                                                                                                                                 |
|------------|----------|-------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| id         | string   | sim                                       | Identificador <b>estável e único</b> (formato caminho, ex.: <code>documentacao/frotas/cadastro-veiculos.md</code> ). <b>NÃO DEVE</b> mudar durante a vida do documento e <b>NÃO DEVE</b> ser reutilizado após deleção. |
| titulo     | string   | sim                                       | Título legível — usado na citação da resposta ao cliente final.                                                                                                                                                        |
| categoria  | string   | sim                                       | Agrupamento lógico (ex.: <code>Frotas</code> ). Usado em metadados e citação.                                                                                                                                          |
| conteudo   | string   | sim (só em <code>GET / docs/{id}</code> ) | Markdown UTF-8, estruturado com cabeçalhos <code># / ## / ###</code> — a estrutura de cabeçalhos define as seções citáveis do índice.                                                                                  |
| updated_at | datetime | sim                                       | ISO 8601 em UTC. Ver regras na seção 4.1.                                                                                                                                                                              |
| deleted    | boolean  | sim                                       | Tombstone explícito. Ver seção 4.5.                                                                                                                                                                                    |
| deleted_at | datetime | quando <code>deleted=true</code>          | ISO 8601 em UTC.                                                                                                                                                                                                       |

### 4. Garantias exigidas do provedor

#### 4.1 `updated_at` — o único sinal de mudança

O consumidor **não re-baixa conteúdo para conferência** no ciclo diário: a decisão de reprocessar um documento vem exclusivamente dos metadados da listagem (seção 6.1). Uma mudança de `conteudo` sem avanço de `updated_at` é, portanto, **invisível** ao pipeline até a reconciliação (seção 6.4) — defasagem silenciosa da base. Esta é a garantia mais sensível do contrato.

- **DEVE** avançar sempre que **qualquer** campo visível mudar: `conteudo`, `titulo`, `categoria` ou `deleted`. Mudança de categoria sem `updated_at` novo é atualização perdida — o índice fica defasado silenciosamente.
- **DEVE** ser monotônico por documento (nunca retroceder).
- **PODE** avançar sem mudança real de conteúdo (ex.: save sem alteração) — o consumidor reprocessa o documento mesmo assim, por projeto (seção 6.1). Aceitável se ocasional; se for sistemático (ex.: rotina que "toca" todos os documentos), reprocessa a base inteira todo dia e **DEVE** ser corrigido na origem.

#### 4.2 Listagem sempre completa

- `GET /docs` **DEVE** retornar todos os documentos vivos + tombstones em retenção, sem nenhum filtro implícito (por categoria, por permissão, por data).
- Se a API não puder garantir a completude da listagem em uma chamada (falha em dependência interna, timeout parcial), ela **DEVE** retornar erro `5xx` — **NUNCA** `200` com lista

parcial. Esta é a cláusula mais importante do contrato: um `200` parcial é indistinguível de "documentos não existem" e envenena a decisão do sync.

### 4.3 Paginação

- Cursor opaco ( `next_cursor` ), `limit` sugerido de 500 itens.
- Uma travessia completa de páginas **DEVE** enumerar cada documento exatamente uma vez (ordenação estável — ex.: por `id` ). Documento alterado durante a travessia **PODE** aparecer com qualquer uma das versões; o próximo ciclo corrige.

### 4.4 Identidade

- `id` imutável. **Renomear/mover** um documento (mudar o caminho) é, para este contrato, **deleção + criação**: o `id` antigo vira tombstone e o novo caminho entra como documento novo.

### 4.5 Tombstones (deleção explícita)

- Documento deletado **DEVE** permanecer na listagem com `deleted=true` por **no mínimo 30 dias** após a deleção (retenção). O job roda diariamente; a folga cobre falhas prolongadas do pipeline sem perder o evento de deleção.
- Após a retenção, o tombstone **PODE** ser expurgado da listagem.
- Deleção **NUNCA** é sinalizada por omissão da listagem.

## 5. Cenários — comportamento esperado em cada um

| # | Cenário                                        | O que a API retorna                                                                                    | O que o pipeline faz                                                                                                                                |
|---|------------------------------------------------|--------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| 1 | <b>Documento novo</b>                          | Aparece na listagem com <code>updated_at</code> de criação, <code>deleted=false</code> .               | Busca conteúdo, chunkeia, embeda, insere no índice.                                                                                                 |
| 2 | <b>Conteúdo alterado</b>                       | Mesmo <code>id</code> , <code>updated_at</code> avançou.                                               | Hash da listagem mudou → busca o conteúdo e substitui todos os chunks do doc em uma transação.                                                      |
| 3 | <b>Só metadado alterado</b> (título/categoria) | <code>updated_at</code> avançou.                                                                       | Hash da listagem mudou → reprocessa o doc (título/categoria entram nos metadados citáveis).                                                         |
| 4 | <b>Sem mudança</b>                             | Listagem idêntica ao ciclo anterior.                                                                   | Hash da listagem igual ao registrado → nem busca o conteúdo. Custo do ciclo $\approx$ zero.                                                         |
| 5 | <b>Save sem mudança real</b>                   | <code>updated_at</code> avançou, conteúdo idêntico.                                                    | Reprocessa o doc mesmo assim — <b>por projeto</b> : o pipeline prefere reproprocessar à toa (custo local de ms) a arriscar perder uma mudança real. |
| 6 | <b>Documento deletado</b>                      | <code>deleted=true</code> na listagem (mín. 30 dias); <code>GET /docs/{id}</code> → <code>410</code> . | Remove os chunks do doc e marca deleção na tabela de controle.                                                                                      |

| #  | Cenário                                                | O que a API retorna                                                           | O que o pipeline faz                                                                                                                                                          |
|----|--------------------------------------------------------|-------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 7  | <b>Documento restaurado</b>                            | <code>deleted=false</code> novamente, <code>updated_at</code> avançou.        | Trata como documento novo/alterado (cenários 1-2).                                                                                                                            |
| 8  | <b>Renomeado/movido</b>                                | Tombstone do <code>id</code> antigo + documento novo no <code>id</code> novo. | Cenário 6 para o antigo + cenário 1 para o novo.                                                                                                                              |
| 9  | <b>Falha interna do provedor</b>                       | <code>5xx</code> explícito. <b>Nunca</b> <code>200</code> parcial.            | Aborta o ciclo com alerta; o índice do dia anterior continua servindo (fail-safe).                                                                                            |
| 10 | <b>Deleção em massa legítima</b> (ex.: reestruturação) | Muitos tombstones de uma vez.                                                 | A trava anti-catástrofe (seção 6.2) aborta e alerta; a execução é liberada manualmente após confirmação humana. Avisar a equipe Azapfy com antecedência evita o falso alarme. |
| 11 | <b>Base legitimamente vazia</b>                        | <code>200</code> com <code>itens: []</code> .                                 | Indistinguível de erro grave — cai na trava 6.2 e exige liberação manual. Cenário esperado apenas em ambiente novo.                                                           |

## 6. Salvaguardas do consumidor (informativo)

Estas regras são do lado do pipeline — constam aqui para o provedor entender o comportamento observável da integração.

### 6.1 Sinal de mudança: hash dos metadados da listagem

O pipeline calcula `sha256(titulo + "\x1f" + categoria + "\x1f" + updated_at + "\x1f" + deleted)` sobre cada item da listagem e compara com o valor registrado no ciclo anterior. **Qualquer** diferença → busca o conteúdo e reprocessa o documento inteiro (chunks + embeddings). Não há comparação de conteúdo no ciclo diário — a decisão vem de um único campo determinístico, calculado sem baixar documento nenhum.

Viés assumido: **falso positivo é barato** (re-embedar um doc custa milissegundos de CPU local), **falso negativo é caro** (o agente responde ao cliente final com informação desatualizada, sem que ninguém perceba). Por isso o pipeline reprocessa na dúvida — e por isso a confiabilidade de `updated_at` (seção 4.1) é a garantia que sustenta todo o incremental: se o campo não avançar numa mudança real, a correção **DEVE** ser feita na origem, não com heurísticas no consumidor.

### 6.2 Trava anti-deleção em massa

Se em um único ciclo a fração de documentos a remover (tombstones novos) exceder **20%** da base conhecida — ou a listagem vier vazia com base não-vazia — o job **aborta sem deletar nada** e emite alerta. Remoções em massa exigem confirmação humana.

### 6.3 Falha nunca degrada o índice

Qualquer erro (rede, `5xx`, resposta malformada) aborta o ciclo inteiro; o índice do dia anterior permanece no ar. O job é idempotente: a reexecução converge sem estado intermediário a limpar.

## 6.4 Reconciliação

Além do incremental diário, o pipeline executa periodicamente (sugestão: **semanal**) uma reconciliação completa — re-leitura de todo o conteúdo e reprocessamento — como rede de segurança para o caso-limite que o incremental não enxerga: mudança de conteúdo sem avanço de `updated_at`. Ocorrências detectadas na reconciliação são reportadas ao provedor como violação da seção 4.1.

---

## 7. Requisitos não-funcionais

---

- **Janela de execução:** o job roda 1× ao dia (madrugada, horário de Brasília). A API **DEVE** estar disponível nessa janela.
  - **Timeout:** respostas em até 30 s por chamada.
  - **Volume:** listagem paginada; documentos individuais de até 1 MB de Markdown (documentos maiores devem ser divididos na origem).
  - **Conteúdo:** o material publicado nesta API alimenta respostas diretas ao cliente final. **NÃO DEVE** conter dados pessoais (LGPD), credenciais ou informação interna não-publicável.
- 

## 8. Versionamento do contrato

---

- Esta é a **v1.0**. Mudanças compatíveis (campo novo opcional) **PODEM** ser feitas sem aviso; mudanças incompatíveis (remover/renomear campo, mudar semântica) exigem acordo prévio entre as partes e nova versão do contrato.
  - Pontos de contato e prazos de comunicação: a definir na assinatura.
- 

Documento gerado pela equipe de engenharia do agente de suporte (Azapfy Zapin). Fonte canônica: `contrato-b-api-documentacao-v1.md` no repositório do projeto.